

Module: Les technologies du Middleware



Cours 4: Message Oriented Middleware (MOM)

FZ. Zahra

f_zahra@esi.dz



- Message Oriented Middleware (MOM).
- Java Message Service –JMS.
- Apache Kafka

Motivation (1)



Couplage fort

- Le client dépend de l'interface du serveur
- Le client doit «connaître» le serveur (Référence)

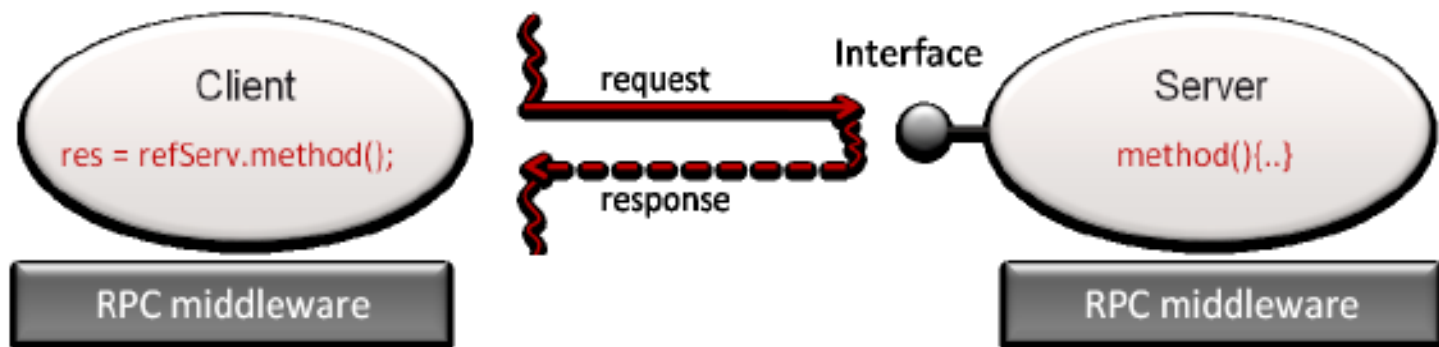
Dépendance temporelle

- Le client et le serveur doivent être simultanément disponibles

Communication synchrone/bloquante (habituellement)

- Le client est bloqué entre l'envoi de la requête et l'arrivée de la réponse

RPC / RMI – Rappel





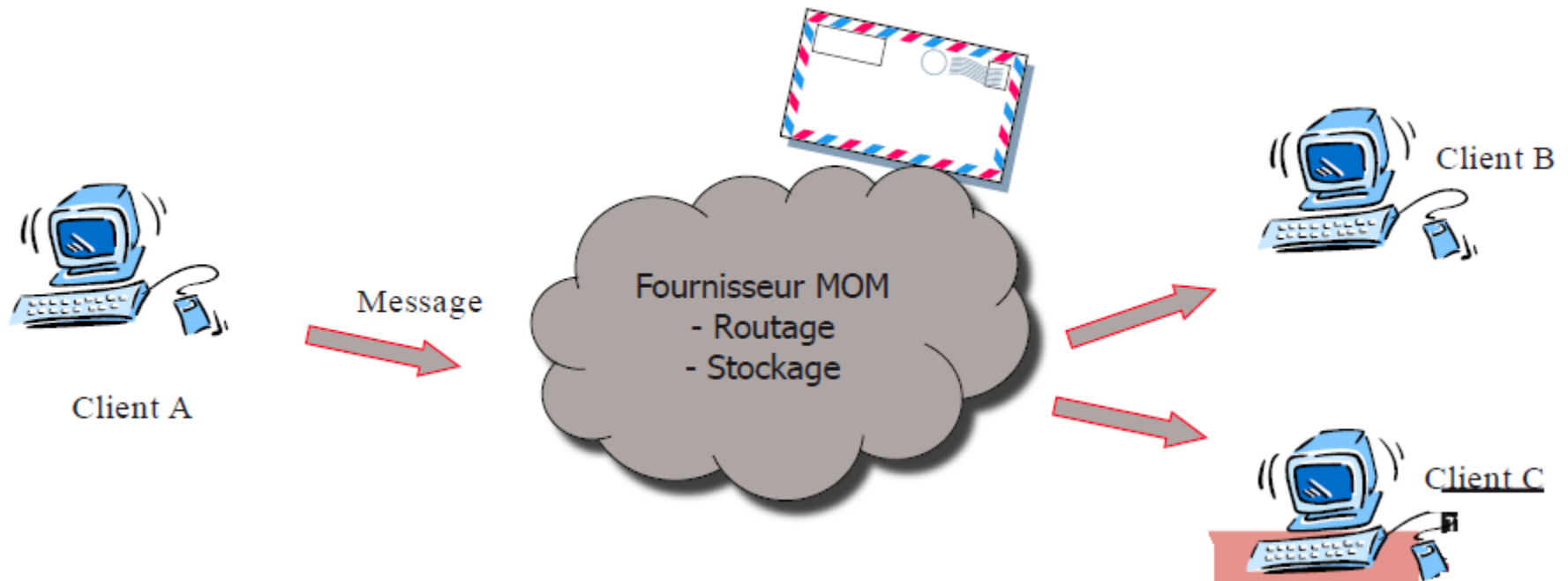
Différentes applications ont des besoins et contraintes différents

- Pas de disponibilité simultanée
 - les différents composants de l'application ne sont pas toujours disponibles simultanément (surtout dans les applications réparties à grande échelle)
- Possibilité de communication asynchrone / non-bloquante
 - La logique métier de l'application permet à un composant d'envoyer des informations à un autre composant et de continuer son exécution sans recevoir une réponse immédiate
- Besoins d'un couplage faible
 - Le développeur veut éviter qu'un composant dépende de l'interface des autres composants ou même de «connaître» les autres composants (Références directes) => remplacement facile

Motivation (3)



Solution → Communication par Message



Message Oriented middleware (MOM)

Intergiciels Orientés Messages (IOM)



Intergiciel Orienté Messages = Message Oriented Middleware

■ Une définition unique:

« modèle de répartition fondé sur l'échange de messages entre les nœuds d'une application répartie ».

■ Les intergiciels orientés messages (MOM) :

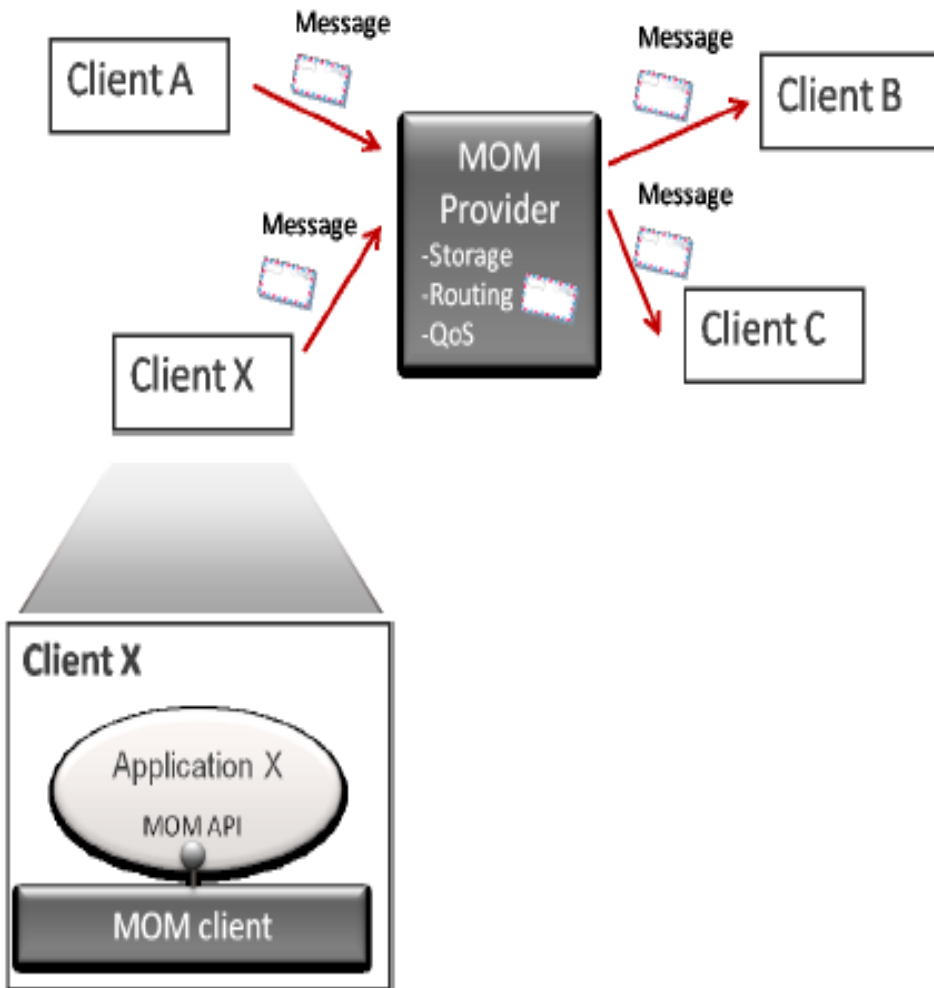
Proposent un modèle simple et fiable pour l'échange de messages dans une application répartie

- Utilisent un des modèles de communication les plus anciens
- Sont utilisés dans les réseaux de grande dimension:
 - Réseaux bancaires
 - Télécommunications
 - Systèmes de réservation, commerce
 - etc



- Garantie de délivrance de messages
- Support des transactions
- Gestion du routage
- Passage à l'échelle
- Support pour la configuration (politiques de QoS)
- Composants faiblement couplés (généralement !)
 - Pas de dépendance d'interface
 - Pas de référence directe
 - Pas de dépendance temporelle – l'exécution de l'émetteur n'est pas interrompue si le destinataire n'est pas disponible
 - Communication asynchrone / non-bloquante (pas de réponse implicite, sauf Ack.)

MOM –Architecture générale



Client –utilisateur du MOM

- Émission et réception de messages

Fournisseur –«provider»

- support du MOM : routage, stockage des messages

Message -support des données transmises

- formats texte, son, image, ..
- QoS-priorité, date de remise, ..



Caractéristiques

- Synchrone vs. Asynchrone
- Persistant vs. Transitoire
- Unicast vs. Multicast
- Mode «push» vs. «pull»
- Patrons de communication :Requête/Réponse, Point à Point, Publication / Abonnement, ...
- Systèmes de routage des messages
- Bibliothèque de traitement de messages
- ...

⇒ Plusieurs modèles de communication par message

- Implantant diverses combinaisons des aspects indiqués auparavant



Synchrone / bloquante

- L'émetteur reste bloqué jusqu'à ce que le destinataire acquitte la réception du message («Acknowledgement»).

Asynchrone / non-bloquante

- L'émetteur continue de s'exécuter après avoir soumis le message pour la transmission.



Communication persistante («persistent»)

- Le serveur MOM conserve le message jusqu'à ce qu'il soit transmis au récepteur; le message n'est jamais perdu ou effacé
- => pas de dépendance temporelle : l'émetteur et le récepteur ne sont pas obligés d'être présents en même temps

Communication transitoire («transient»)

- Le serveur MOM conserve le message seulement pendant l'exécution simultanée de l'émetteur et du récepteur
- => dépendance temporelle : l'émetteur et le(s) récepteur(s) doivent être présents en même temps



«Unicast»

- Le message est envoyé à un seul destinataire.

«Multicast» -Diffusion (ou group)

- Le message est distribué à plusieurs destinataires.



Mode «push»

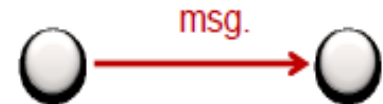
- L'émetteur envoie le message au récepteur
- Ex. en appelant une méthode de «callback» du récepteur

Mode «pull»

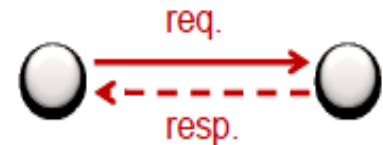
- Le récepteur va chercher le message chez l'émetteur
- Ex. :
 - périodiquement;
 - en restant bloqué jusqu'à ce qu'un message devienne disponible;
 - sur notification de l'émetteur («push» et «pull»)



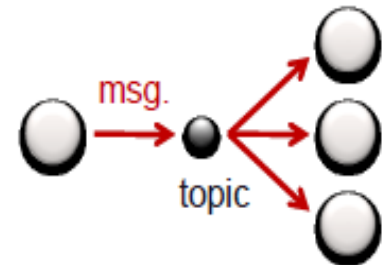
■ Point à Point (« Point to Point »)



■ Requête / Réponse



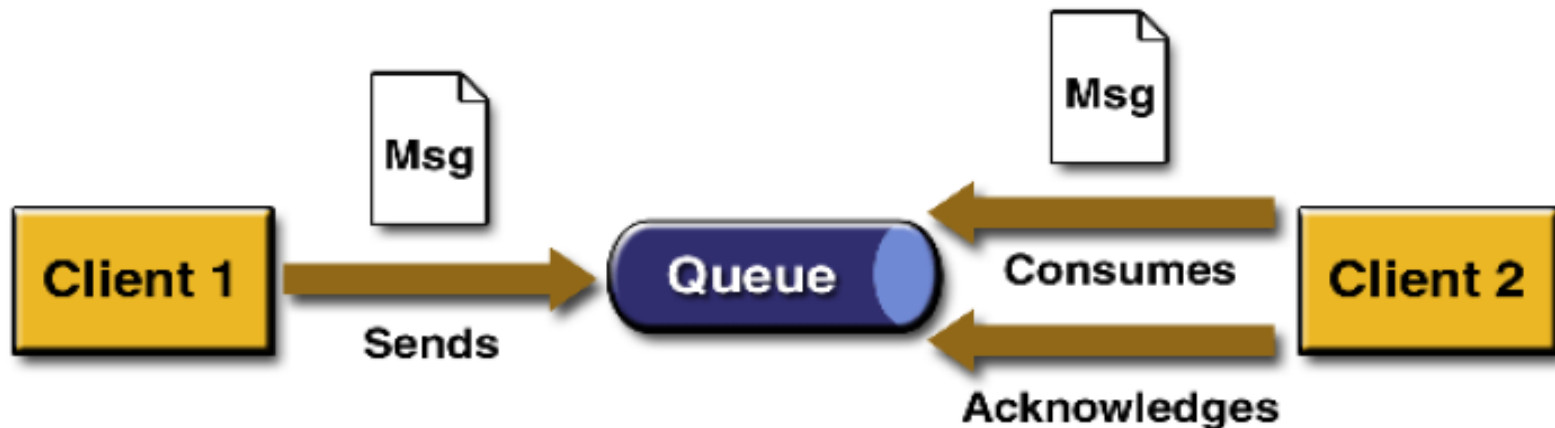
■ Publication/Abonnement (« Publish/Subscribe »)



■ ...

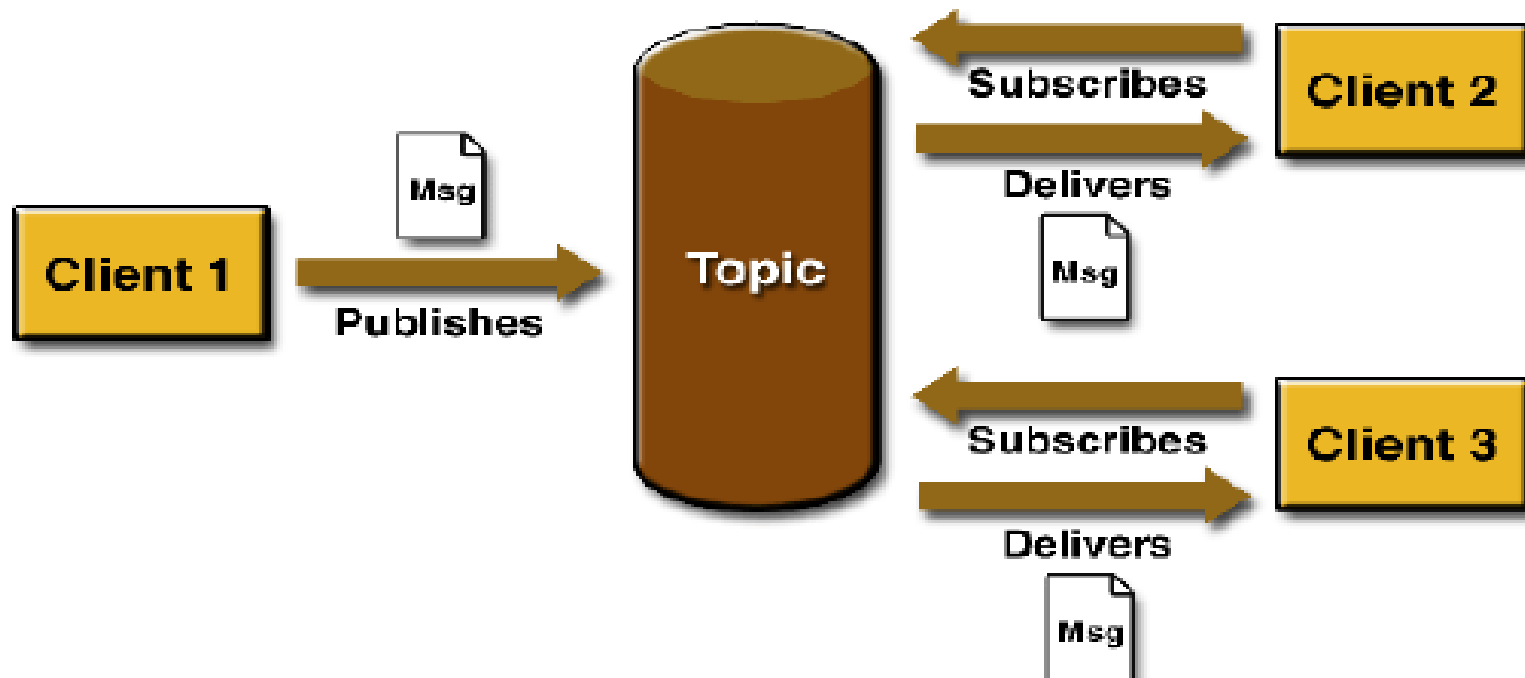


- Chaque message est à destination d'un unique lecteur, stocké dans une *file*(«queue») dédiée jusqu'à ce que le destinataire lise le message
- Chaque message a un consommateur unique
- Il n'y a pas de dépendance entre émetteur et destinataire du message. Le lecteur peut lire le message “plus tard”
- Le destinataire peut acquitter les messages reçus





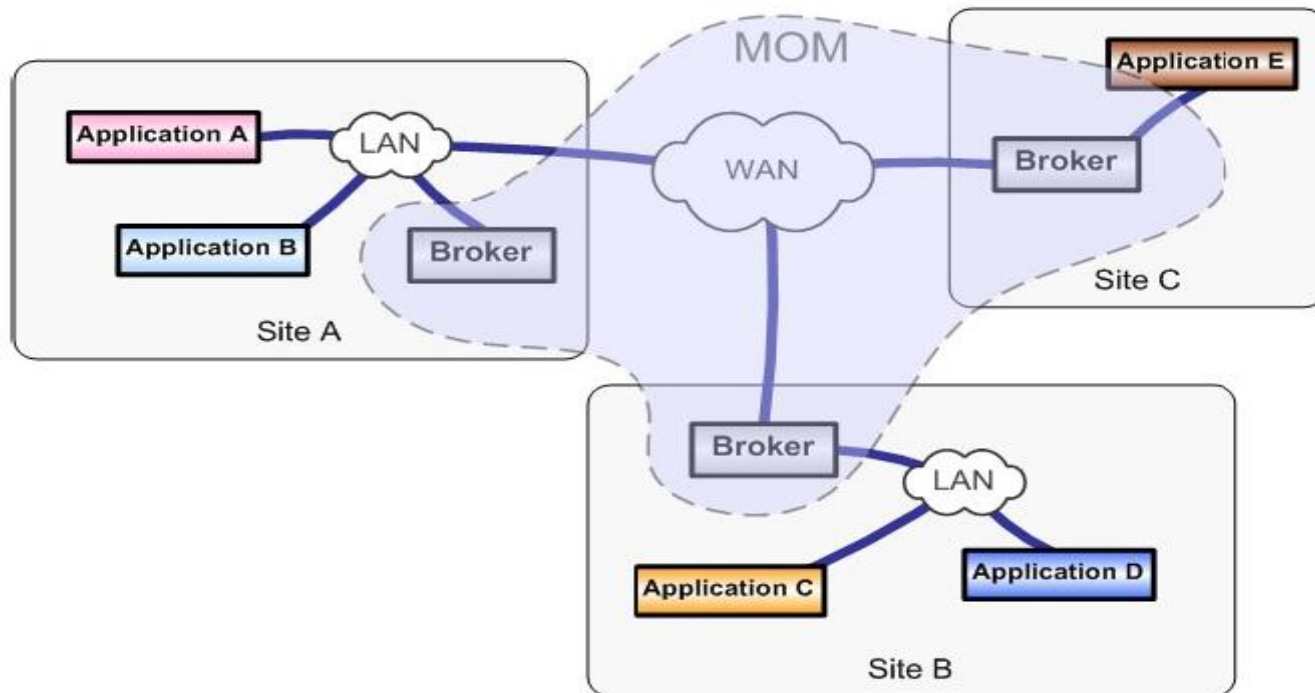
- Le sujet («topic») gère l'envoi de messages pour un ensemble de lecteurs abonnés à ce topic.
- Dépendance temporelle : un client ne peut lire un message qu'après s'être abonné à un topic, et doit continuer à être actif pour les recevoir.
- Cette contrainte peut être relâchée via les «durable subscription» : on continue de recevoir les messages entre deux instants où le client «disparaît» (spécification JMS)



Topologies pour le Fournisseur de MOM



- Différentes topologies possibles pour le Message Broker
 - Centralisée
 - Décentralisée
 - Hybride
- Chaque produit MOM peut utiliser une ou plusieurs topologies



Architecture centralisée



■ Serveur central de gestion de messages

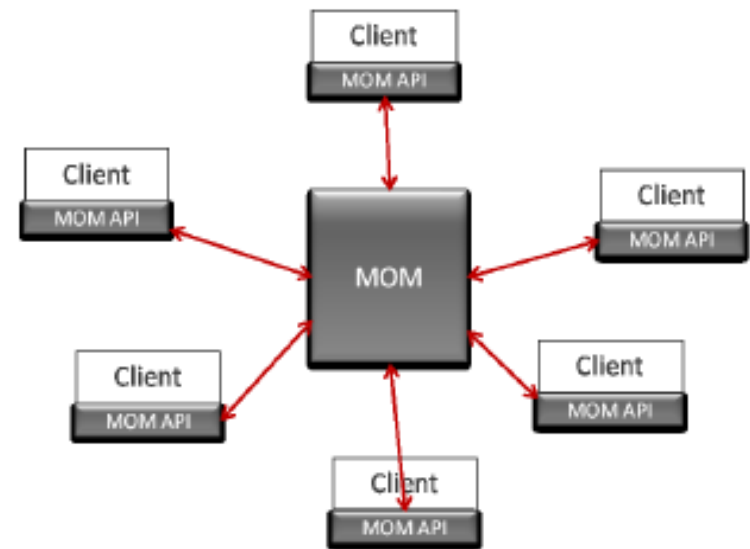
- Routage, stockage, transactions, ..

■ Avantages

- Découple les clients, qui ne voient que le serveur - l'ajout ou l'enlèvement d'un client n'a pas d'impact sur les autres clients

■ Inconvénients

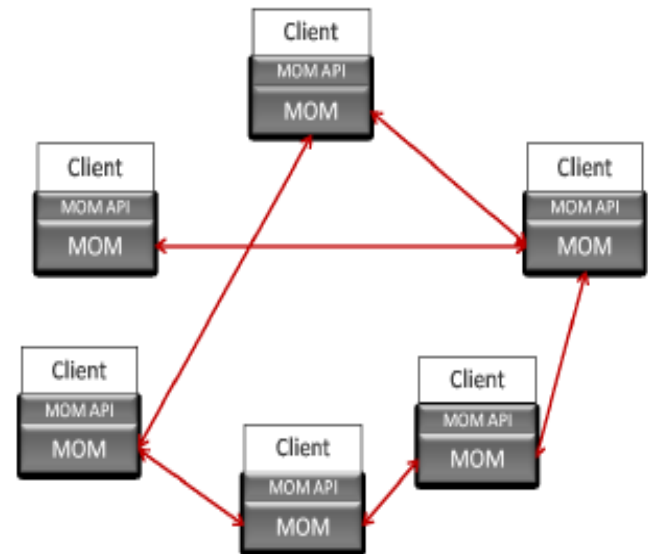
- Engendre un trafic réseau important
- Crée un goulot d'étranglement:
 - Point unique d'échec (« unique point of failure »)
 - Problèmes de passage à l'échelle
 - Possibilité de performance réduite (latence)



Architecture décentralisée



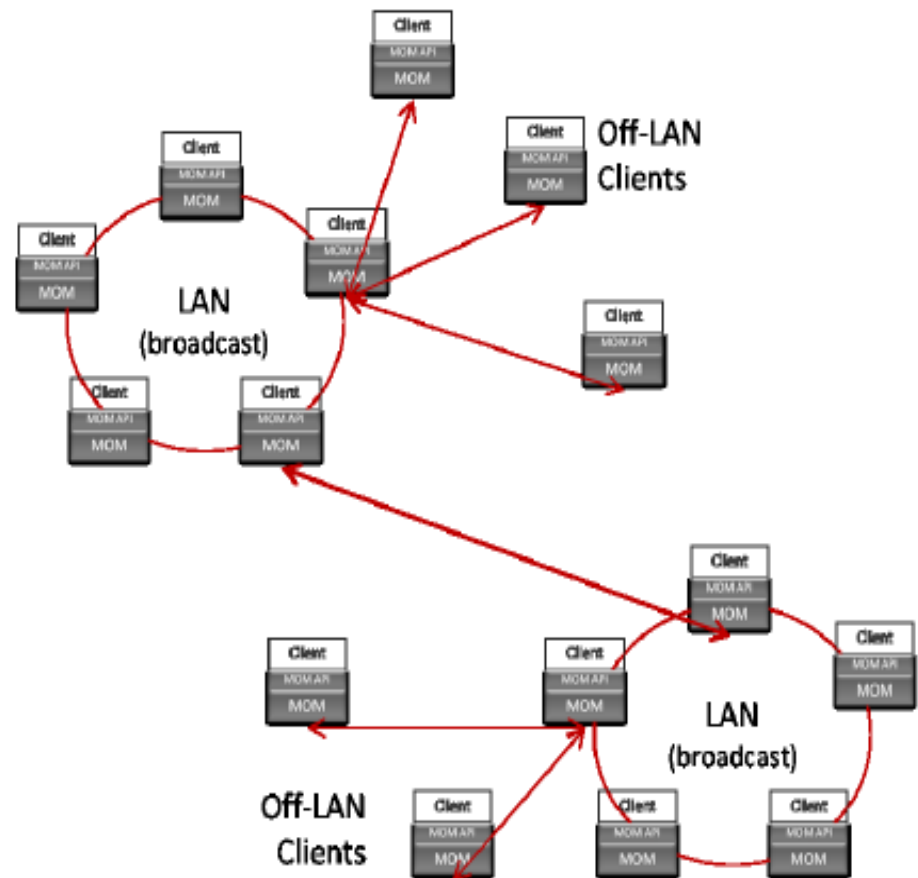
- Une instance de MOM est installée chez chaque Client - pas de serveur central
 - Stockage, transactions, sécurité, ...
 - Routage de messages
 - Ad-hoc – entre les MOMs
 - Basé sur le protocole réseaux existant - ex. IP-Multicast
- Avantages
 - Distribution des fonctions du MOM entre les serveurs – ex. persistance, sécurité, ...
- Inconvénients
 - Problèmes potentiels d'interopérabilité – ex. différents vendeurs ou versions de MOM
 - Clients plus lourds, duplication des fonctions



MOM –topologie hybride



- Combine les deux architectures précédentes
- Combine les avantages et les inconvénients





Administration du MOM

- Déploiement
 - Position des nœuds
 - Allocation des ressources
- Configuration des files de messages
 - Taille des files
 - Persistance
 - Filtrage des messages
- Outils d'aide à l'administration

Configuration des clients

- Point d'accès au MOM
 - Authentification
 - Établissement de connexions
- Mode d'émission/réception des messages
 - Type de connexion
 - Priorités d'accès au MOM
 - Filtrage en réception



JMS: Java Message Service API



MOM –besoins de standardisation

■ Un modèle de répartition - Une définition unique

- « *modèle de répartition fondé sur l'échange de messages entre les nœuds d'une application répartie* »

■ Multiples solutions d'implantation

- Différentes sémantiques et services offerts
 - Envoi « à l'aveugle » des messages ou support de l'acquittement; support des transactions; ...
 - Gestion de la mobilité, support pour la reconfiguration dynamique, ...
- Différentes architectures et implantations
 - Communication à base de TCP/IP, IP multicast, SSL, HTTP, une sous-couche RPC, ...
 - Diverses implantations pour les files et les topics
 - Diverses topologie supportées – centralisée, décentralisée, hybride
- => Besoins de standardisation
 - Ex: Java Message Service (JMS), OMG COS Events/Notification, WS-Reliable Messaging, AMQP, ...



MOM -efforts de standardisation

- Jusqu'à ~2001, peu ou pas d'efforts de normalisation
 - Une API par vendeur
 - Conceptions différentes (ex. utilisation des ressources)
 - Fonctionnalités différentes

- Difficultés
 - Limitation de l'interopérabilité (critique)
 - Problèmes de maintenance et d'évolutivité

- Évolutions
 - **Java Messaging Service (JMS)** - une API standard pour le client
 - **CORBA COS Notification** - une API pour le client, description de l'infrastructure (objets et API)
 - **Advanced Message Queueing Protocol (AMQP)** – un standard ouvert pour l'interopérabilité d'intérgiciels à message écrits en différents langages et pour différentes plates-formes

Sun JMS: Java Message Service



- **Spécification d'un API pour les MOM**
 - Intégré à J2EE 1.3 ++, couplage avec les EJB (Message-driven bean)
 - Première spécification d'un MOM publiquement accessible
 - Implémentée par les principaux MOM
 - Adaptable à d'autres langages (C++, Ada)
 - Peu restrictive: synthèse des MOM existants => autorise plutôt qu'interdit
 - **JMS : spécification 1.1**
 - Spécification pour le client
 - P-t-P, Pub/Sub, call-backs,
 - Filtrage (syntaxe à la SQL) et transactions
 - Type de messages
 - Ne spécifie pas l'infrastructure
 - Protocole, représentation, transport
 - Processus de configuration
 - Gestion des erreurs, de la reprise sur pannes
 - Interfaces d'administration
 - Sécurité
-



JMS --Lien avec d'autres API Java

- JNDI: annuaire
- JTA: transaction

Fournisseurs JMS

- **Sun Java System Message Queue (Sun → Oracle)**
 - JMS intégré au serveur d'entreprise de Sun (GlassFish)
 - <http://www.oracle.com/us/products/middleware/application-server/oracle-glassfish-server/index.html>
- **MQ JMS (IBM)**
 - Un des leaders sur ce marché
 - http://www7b.software.ibm.com/wsdd/library/techtip/0112_cox.html
- **WebLogic JMS (BEA → Oracle)**
 - Serveur d'entreprise, peut supporter JMS
 - http://download.oracle.com/docs/cd/E13222_01/wls/docs92/messaging.html
- **JMSCourier (Codemesh)**
 - Application C++ et JMS
 - <http://www.codemesh.com/en/AlignTechnologyCaseStudy.html>
- **TIBCO** <http://www.tibco.com/products/soa/messaging>
- **Fiorano Software** <http://www.fiorano.com>
- **JRUN Server** <http://www.allaire.com>
- **GemStone** <http://www.gemstone.com>
- **Nirvana** <http://www.pcbsys.com>
- **Oracle** <http://www.oracle.com>
- ...
- **Vendor lists**
 - <http://www.techspot.co.in/2006/06/jms-vendors.html>
 - <http://adtmag.com/articles/2003/01/31/jms-vendors.aspx>

JMS Client –Mais comment obtenir la Fabrique de Connexions et les Destinations?



■ L'administrateur du MOM:

- Crée les Usines de Connexions (FC) et les Destinations (D)
- Ajoute les Objets Administrés dans l'annuaire JNDI (opération Bind)

à l'aide d'un Outil Administratif

■ Les Clients JMS:

- Utilisent l'annuaire JNDI afin d'obtenir les Objets Administrés – FC, D
- Utilisent les Fabriques afin d'obtenir des connections logiques au MOM
- Utilisent les Destinations afin d'envoyer/recevoir des Messages

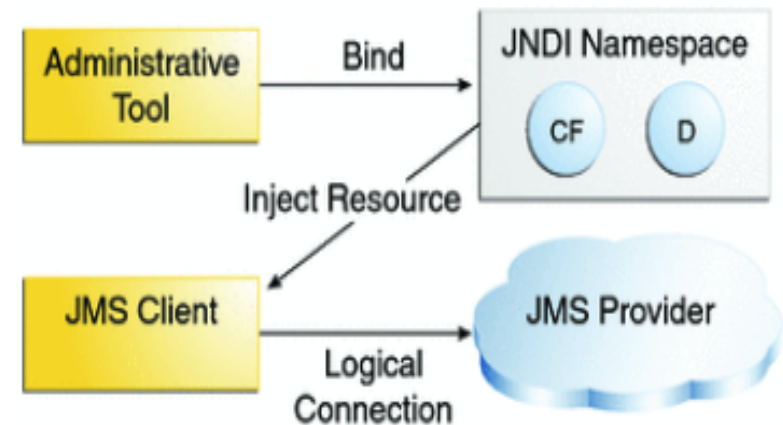


Image from Oracle's JMS tutorial

<http://download.oracle.com/javaee/6/tutorial/doc/bncdx.html>

JMS Architecture



■ JMS Client – utilise l'API JMS

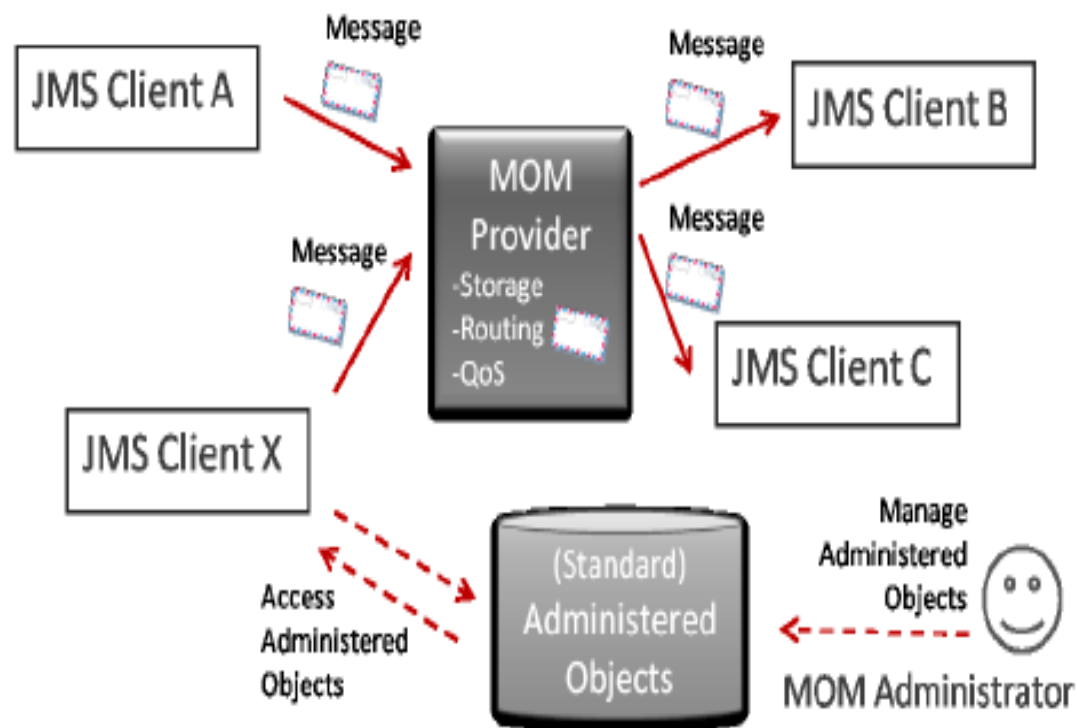
- Pour envoyer/recevoir des messages
- Pour les « Objets Administrés »

■ JMS Provider – impl. l'API JMS

- Gestion des files et topics
- Transmission des messages
- Configuration globale du MOM
- Source d'hétérogénéité –
ex. mécanismes d'administration, protocole utilisé, gestion des erreurs, ...

■ JMS Message – objets contenant l'information communiquée

- Un en-tête
- Des propriétés (optionnel)
- Un corps (optionnel)



■ Objets Administrés

- Fabriques de connexions
- Destinations: files et topics



- *Attention à l'utilisation des termes « synchrone » & « asynchrone »!*
- Réception **Synchrone** - mode « pull », bloquant
 - Le consommateur récupère explicitement le message depuis la Destination en appelant la méthode `receive`.
 - Cette méthode bloque jusqu'à ce qu'un message soit disponible, ou qu'un délai expire.
- Réception **Asynchrone** – mode « push », avec dépendance temporelle
 - Le consommateur enregistre un « message listener » auprès de la Destination ciblée
 - Lorsqu'un message arrive, le fournisseur JMS délivre le message au « message listener » en appelant la méthode `onMessage`.



■ Une Usine de Connexions – « Connection Factory »

- Prend en charge la connexion avec le fournisseur JMS (MOM).
- Encapsule les paramètres de connexion mis en place par l'administrateur.

■ Une Session

- Est un contexte mono-tache
- Utilisé pour l'émission et la réception de messages
- Gère plusieurs consommateurs et producteurs de message

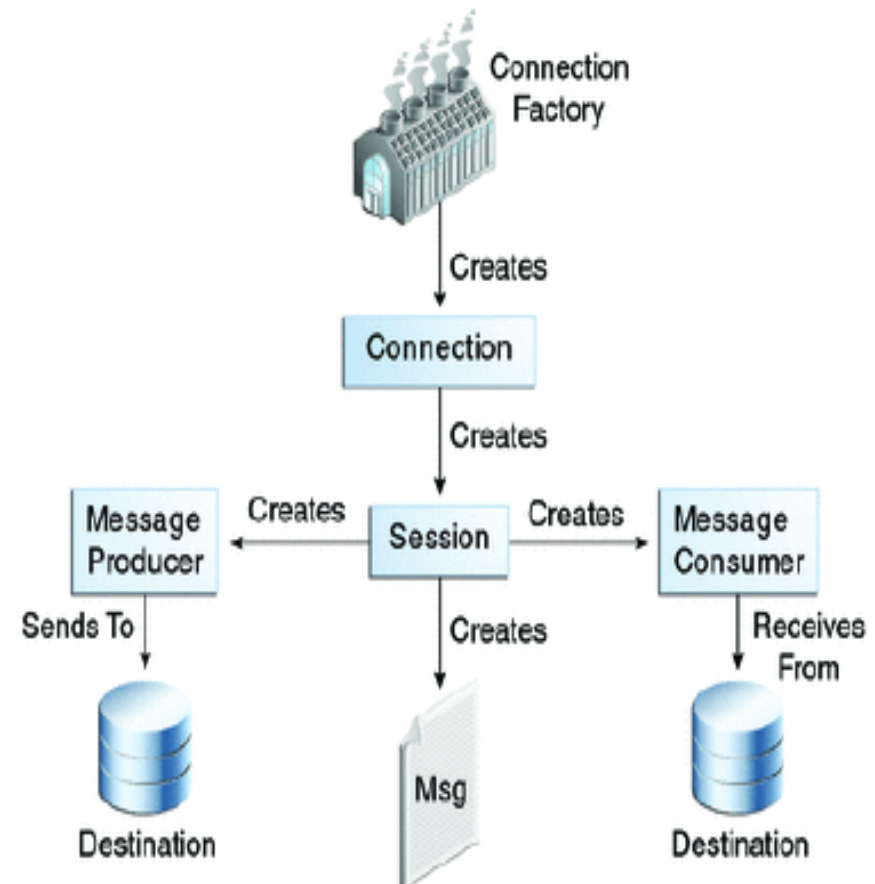
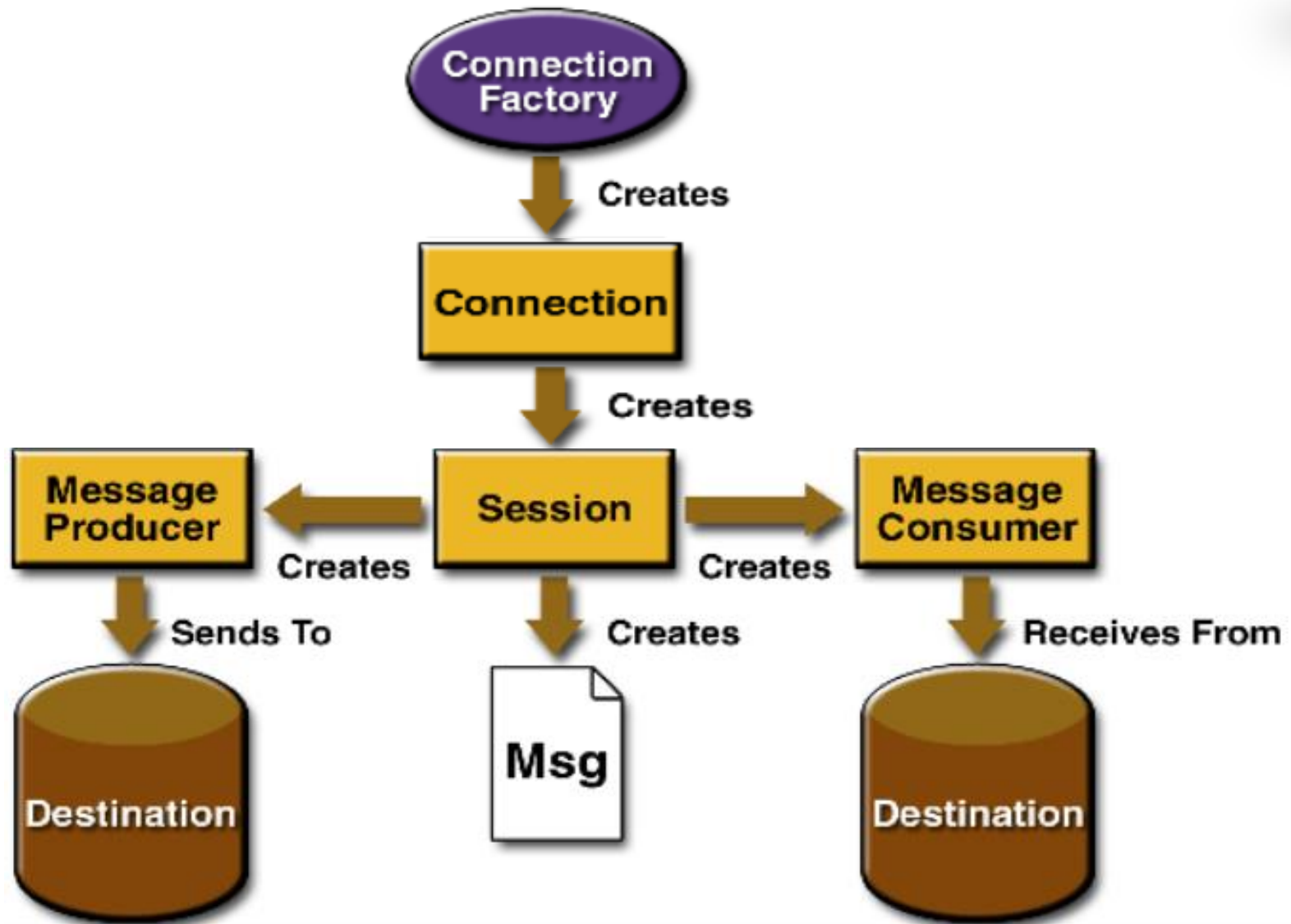


Image from Oracle's JMS tutorial

<http://download.oracle.com/javaee/6/tutorial/doc/bnceh.html>

JMS --Lien avec d'autres API Java



Message JMS



■ Trois parties: entête, propriétés, corps

■ En-tête du message - identification et routage du message

- Paires (nom, valeur):

```
JMSDestination, JMSDeliveryMode, JMSMessageID, JMSTimestamp, JMSExpiration,  
JMSRedelivered, JMSPriority, JMSReplyTo, JMSCorrelationID, JMSType
```

■ Propriétés (optionnel) – spécifiques à l'application, utilisées pour filtrer les messages

```
Ex - sender: message.setStringProperty("Username", "John Doe");
```

```
Ex - receiver: TopicSubscriber sub =  
                session.createSubscriber(chatTopic, "Username != 'William'"); //avec filtre!
```

■ Corps (optionnel) - contient les données de l'application

```
Ex: message.setText(MSG_TEXT + " " + (i + 1));
```

- TextMessage: chaîne de caractères
- MapMessage: ensemble de paires (nom, valeur)
- ByteMessage: flux d'octets
- StreamMessage: flux de valeurs
- ObjectMessage: objet sérialisable

Message JMS



Type de message	Description
Message	Classe de base. Ce type de message est utilisé pour la notification d'événements et ne possède pas de contenu.
BytesMessage	Le contenu est stocké sous la forme d'un tableau d'octets. Ce type de message est utile pour l'échange de données dans un format natif de l'application et lorsque JMS est utilisé comme transport entre deux systèmes, où le client JMS ne connaît pas le type de charge de message. Utilisez ce type de message pour transmettre des messages XML afin de vous assurer que le message est transmis efficacement et qu'il n'est pas soumis à une conversion de données inutile.
TextMessage	Les données sont stockées sous forme de chaîne. Ce type de message est utile pour l'échange de messages texte simples.



Un message de flux est une séquence de types Java™ primitifs. L'objet message suit l'ordre et les types de ces primitives dans le flux. Des règles de conversion formelles s'appliquent ; par exemple, une exception est émise si une application JMS tente de lire une valeur double en tant que valeur courte. Pour obtenir la liste complète des règles de conversion, voir [Java Message Service Specification](#) version 1.1 ou 2.0 .

21ABCDEFGH32.345 est un exemple de charge StreamMessage . Il comprend les trois zones suivantes:

StreamMessage

- Un entier, 21
- Une chaîne, ABCDEFGH
- Une variable flottante, 32.345

Si la structure de données est inconnue, la méthode générique `readObject()` peut être utilisée pour renvoyer l'objet suivant dans le flux. Si la structure des données est connue, le client JMS peut être spécifique au type d'objet auquel il accède.

Message JMS



MapMessage

Le contenu d'un MapMessage est stocké sous la forme d'un ensemble de paires nom-valeur. Le nom est défini en tant que chaîne et la valeur est saisie. Le message MapMessage est utile pour distribuer des données accessibles par clé qui peuvent passer d'un message à l'autre.

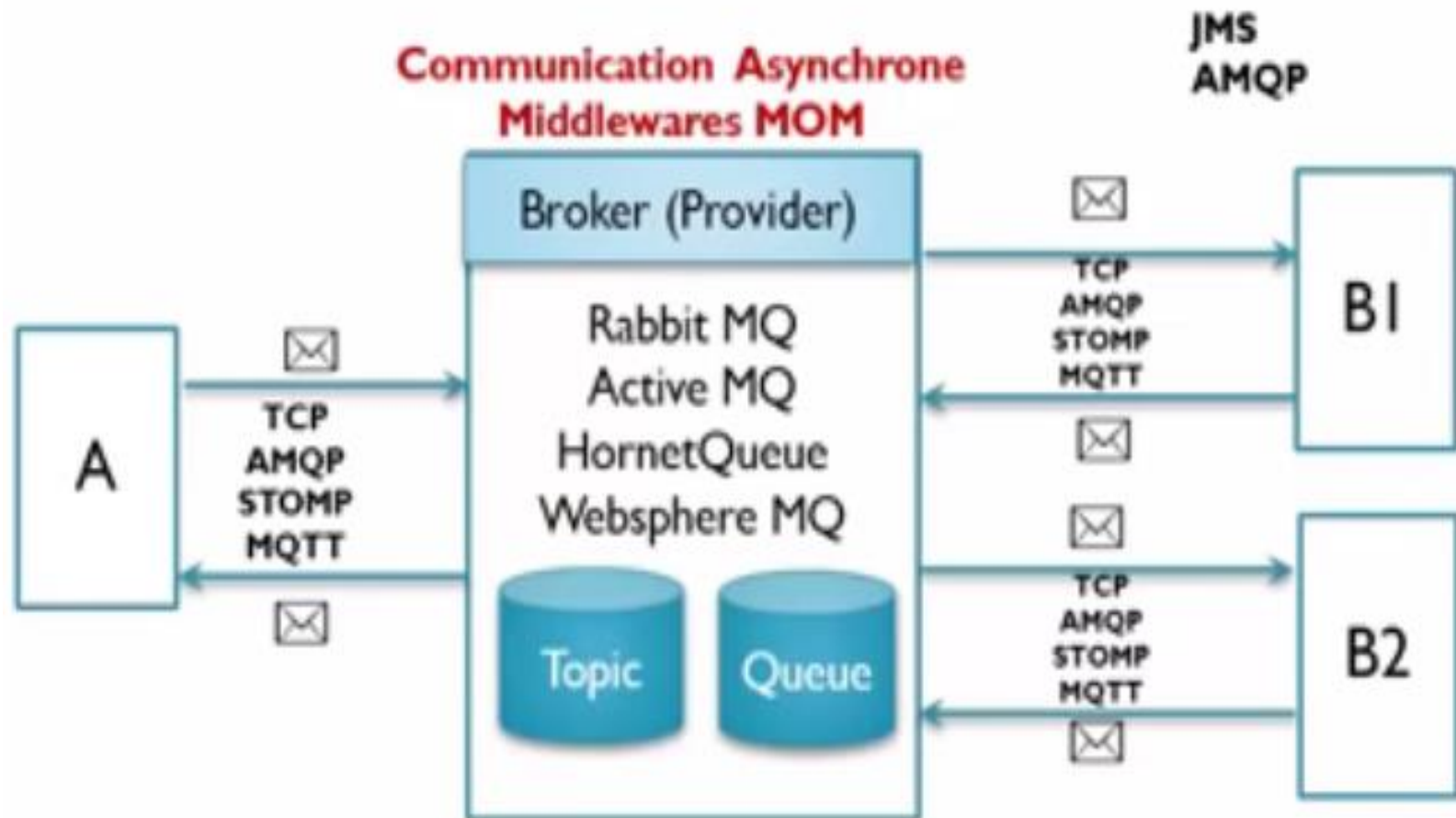
NumberOfCopies:5 est un exemple de charge MapMessage, où NumberOfCopies est la clé et 5 la valeur.

Les données sont accessibles à l'aide de `getMapNames()`, qui renvoie un objet Enumération Java. Il est possible d'itérer via MapMessage en utilisant `hasMoreElements()` pour extraire les paires nom-valeur mappées.

ObjectMessage

Le message Objet contient un objet Java sérialisable comme contenu. Il est utile pour l'échange d'objets Java.

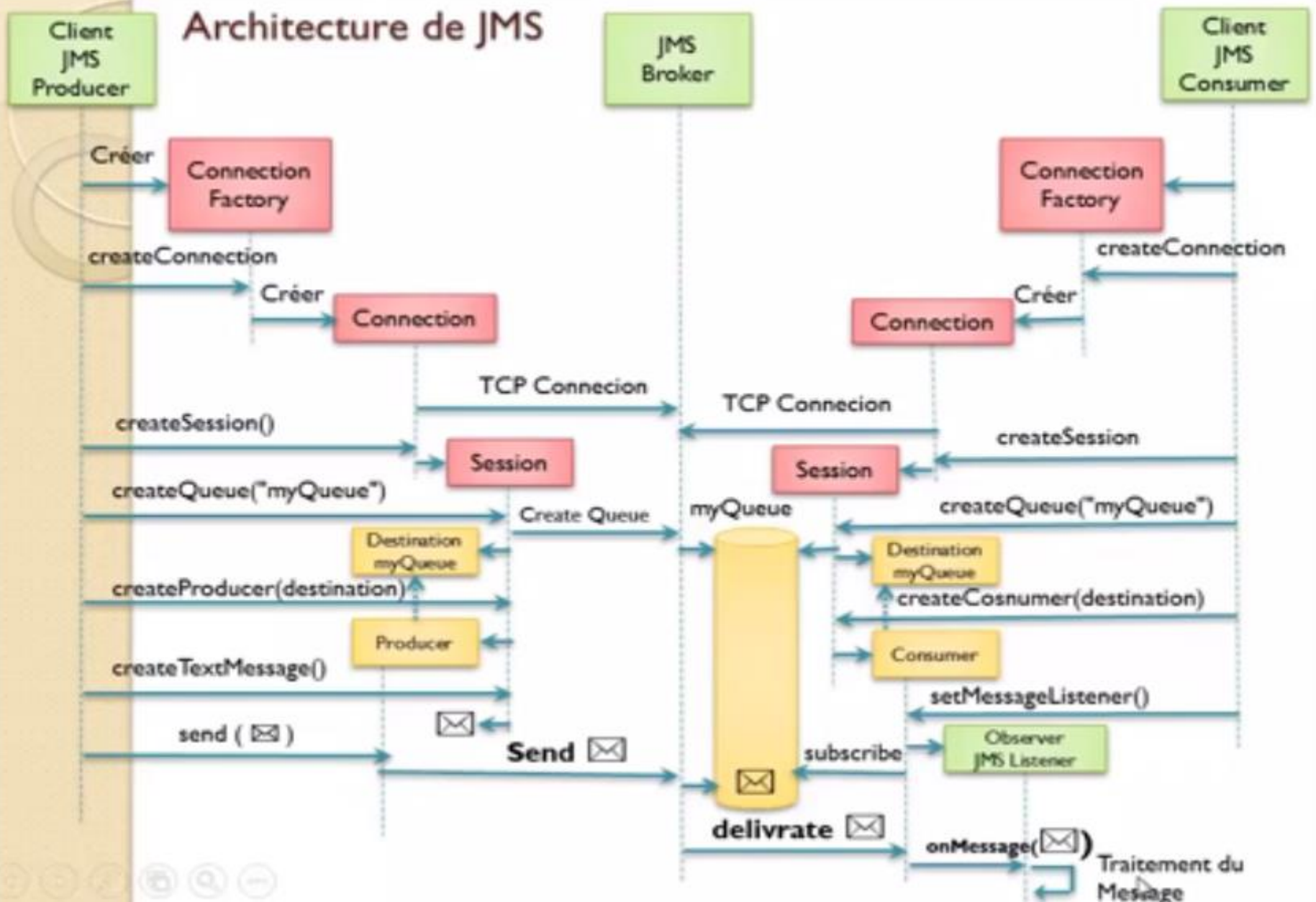
JMS --Lien avec d'autres API Java

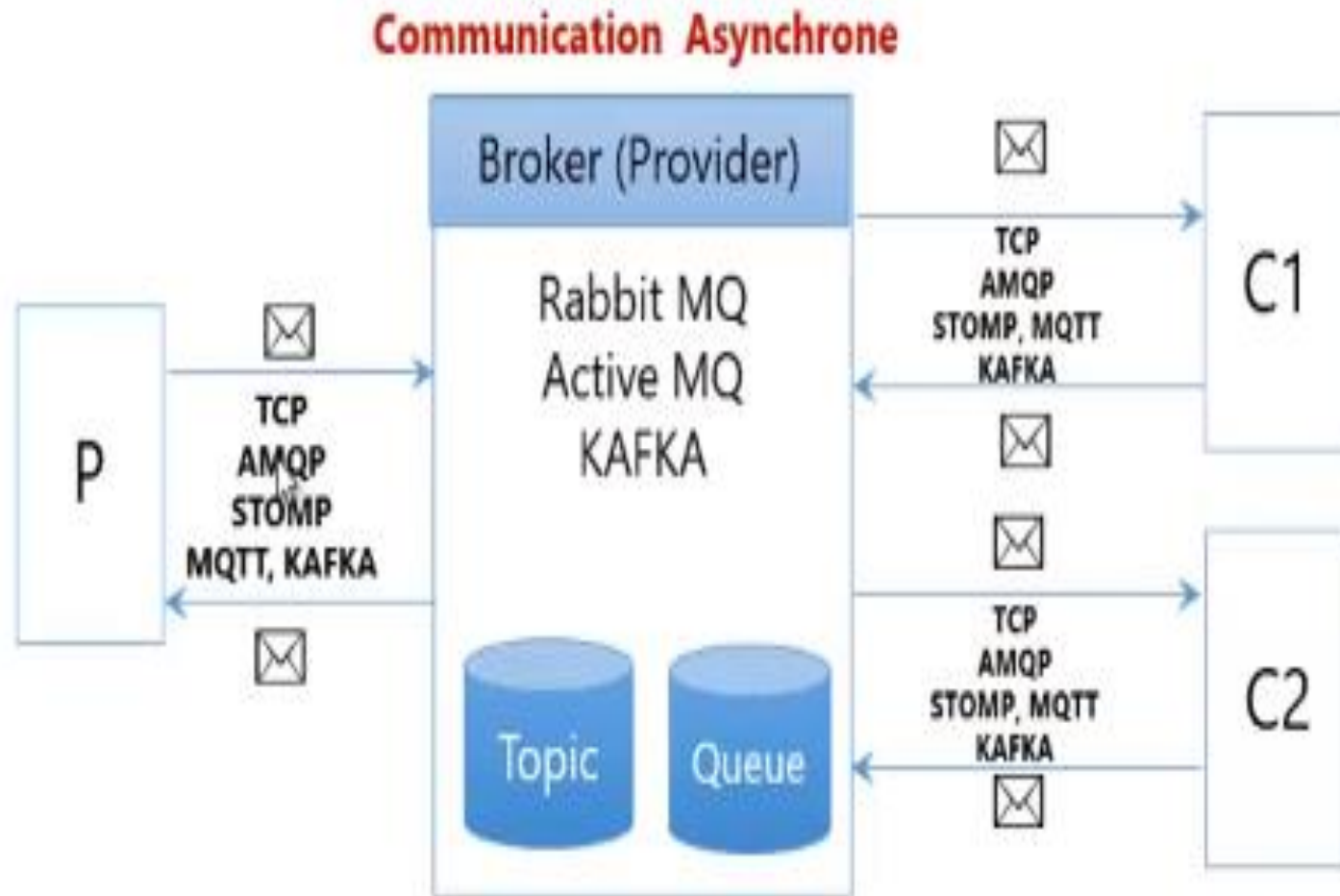


JMS : Architecture dynamique

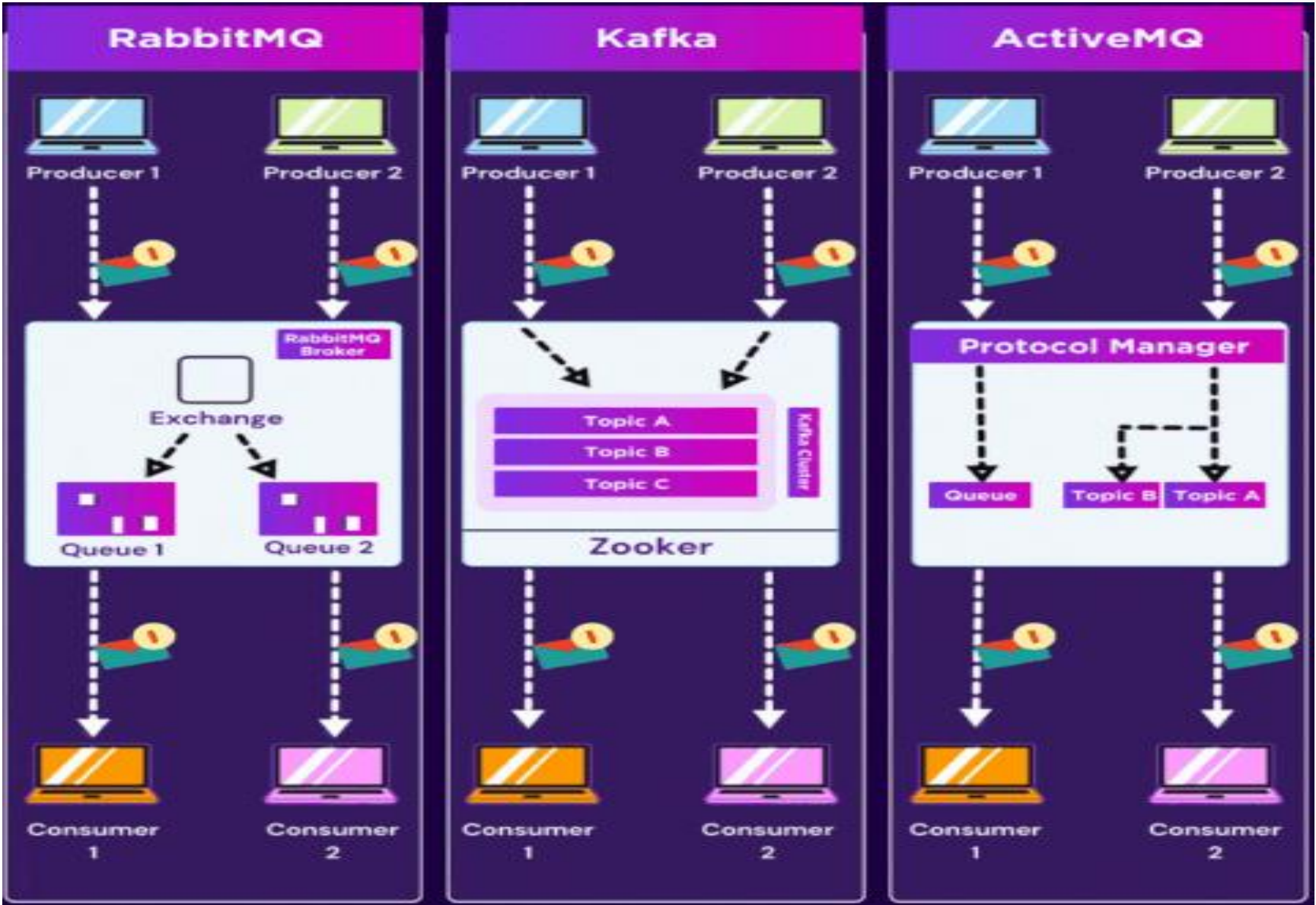


Architecture de JMS





ActiveMQ VS RabbitMQ VS Kafka





Apache Kafka est une plateforme distribuée pour les flux d'événements , Basé sur publish-subscribe:

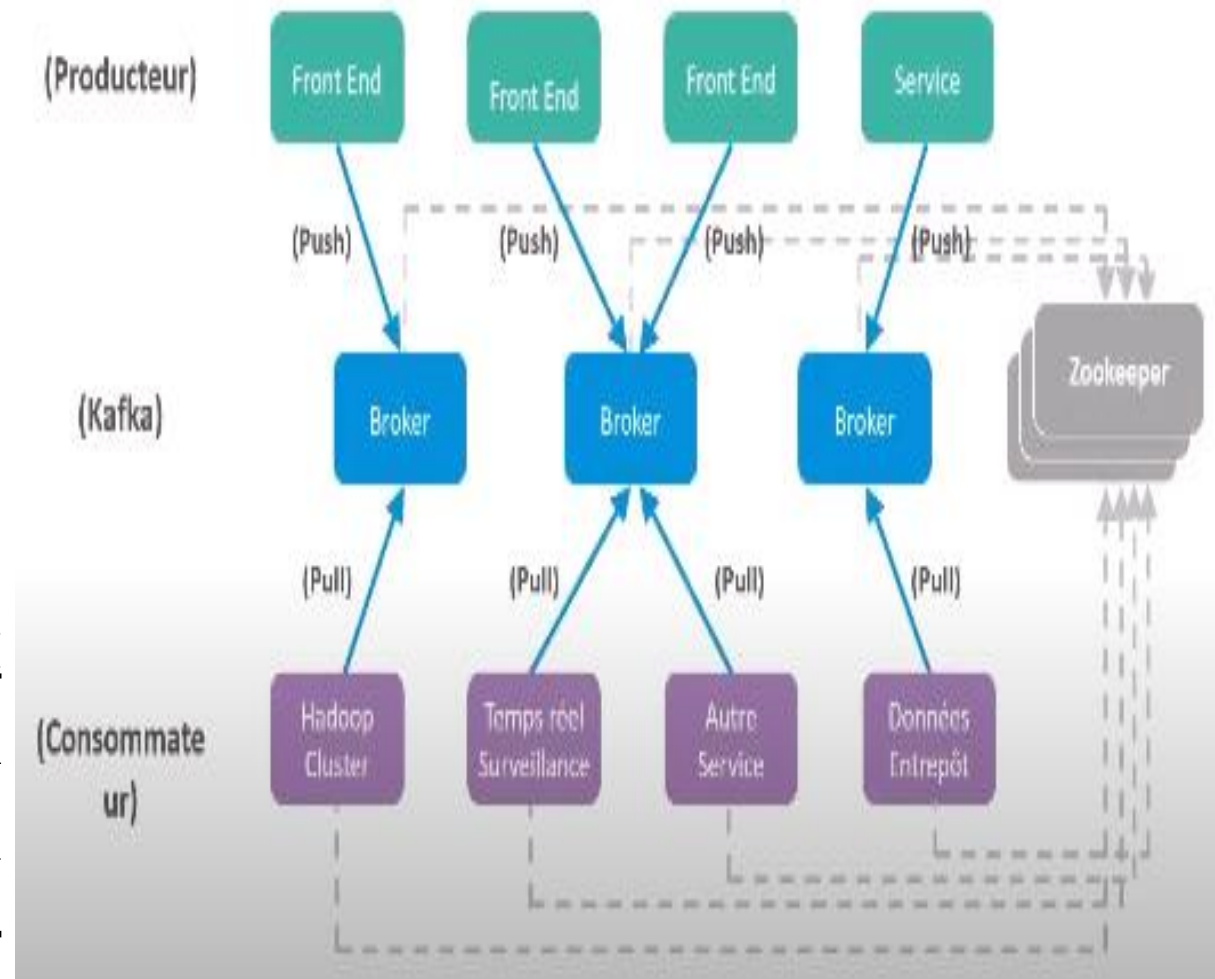
- ❑ S'agit d'un système de messagerie distribuée open-source initialement développé par **LinkedIn**, puis adopté par la Fondation **Apache**.
- ❑ Conçu pour gérer **le flux massif de données en temps réel** de **manière durable**, évolutive et distribuée.
- ❑ Souvent utilisé pour la construction de pipelines de données, la **diffusion de données en continu**, le traitement d'événements en **temps réel**



Kafka: Architecture et APIs

L'architecture de Kafka comprend **cinq API** :

- ✓ **Le Producer API** est une API cruciale qui permet à une application de **publier un flux d'enregistrements** dans un ou plusieurs Topic ;
- ✓ **Le Consumer API** permettant à une application de **s'abonner à un ou plusieurs Topic** et de traiter leur stream ;





- ✓ **Le Streams API** permet le traitement, la transformation et l'**agrégation de données d'événements en temps réel** à partir de topics (sujets) d'entrée vers des topics (sujets) de sortie.
- ✓ **L'API Connect** est un composant important, car elle permet de construire et d'exécuter des connecteurs réutilisables (des producteurs et des consommateurs) qui connectent des topics kafka à des applications et des systèmes de gestion de données. Kafka fournit des APIs clients aux différentes langages de programmation (Java, C++, python, php, .net, ..etc)
- ✓ **L'API Admin** permet aux utilisateurs de **gérer et d'inspecter les sujets**, les brokers, les acls ainsi que tous les autres objets Kafka.



1. Ada Diaconescu, Cours : **MOM -Message Oriented Middleware &JMS - Java Message Service** .
2. Message Oriented Middleware (MOM) principe mise en oeuvre et outil Open source, livre blanc, Smile, 2009.
3. Jérôme Hugues, **Java JMS: architecture des Intergiciels Orientés Messages, 2010.**
4. **Cours et TPs** de professeur Mohamed Youssfi.